Record - Week 05

Topic: Full Stack Integration (MERN)

Name: _____ Roll No: _____ Date: _____

Aim

To integrate a React frontend with an Express and MongoDB backend and build the complete ExpenseWise application.

Software Requirements

Node.js 18+, MongoDB, React 18, Vite, Postman, VS Code

Procedure

1. Create the server folder: install express, cors, dotenv and mongoose.
2. Define the Expense model and the four REST routes in server.js.
3. Create the client with Vite and write App.jsx (form, list, total).
4. Start MongoDB and the server, then start the client with npm run dev.
5. Add and delete expenses in the UI and inspect the requests in the Network tab.

Program

server/server.js

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");

const app = express();
app.use(cors());
app.use(express.json());

const Expense = mongoose.model(
  "Expense",
  new mongoose.Schema(
    {
      title: { type: String, required: true, trim: true },
      amount: { type: Number, required: true, min: 0.01 },
      category: { type: String, default: "Other" },
    },
    { timestamps: true }
  )
);

app.get("/api/expenses", async (req, res) => {
  res.json(await Expense.find().sort({ createdAt: -1 }));
});

app.post("/api/expenses", async (req, res) => {
  try {
    const expense = await Expense.create(req.body);
    res.status(201).json(expense);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

app.put("/api/expenses/:id", async (req, res) => {
  try {
    const expense = await Expense.findByIdAndUpdate(req.params.id, req.body, {
      new: true,
      runValidators: true,
    });
    if (!expense) return res.status(404).json({ error: "Expense not found" });
    res.json(expense);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

app.delete("/api/expenses/:id", async (req, res) => {
  try {
    await Expense.findByIdAndDelete(req.params.id);
    res.json({ message: "deleted" });
  } catch (err) {
    res.status(400).json({ error: "Invalid expense id" });
  }
});

mongoose
  .connect(process.env.MONGO_URI)
  .then(() => app.listen(process.env.PORT || 5000, () => console.log("API running")))
  .catch((err) => console.error(err));
```

server/package.json

```
{
  "name": "mern-server",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": { "start": "node server.js", "dev": "node --watch server.js" },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "mongoose": "^8.4.0"
  }
}
```

server/.env.example

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/expense_tracker
```

client/src/App.jsx

```
import { useEffect, useState } from "react";

const API = "http://localhost:5000/api/expenses";
const CATEGORIES = ["Food", "Travel", "Bills", "Shopping", "Other"];

export default function App() {
  const [expenses, setExpenses] = useState([]);
  const [title, setTitle] = useState("");
  const [amount, setAmount] = useState("");
  const [category, setCategory] = useState("Food");
  const [error, setError] = useState("");

  const load = async () => setExpenses(await (await fetch(API)).json());

  useEffect(() => { load(); }, []);

  const add = async () => {
    if (!title.trim() || !(Number(amount) > 0)) {
      setError("Enter a title and a positive amount");
      return;
    }
    setError("");
    await fetch(API, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ title, amount: Number(amount), category }),
    });
    setTitle("");
    setAmount("");
    load();
  };

  const remove = async (id) => {
    await fetch(`${API}/${id}`, { method: "DELETE" });
    load();
  };

  const total = expenses.reduce((sum, e) => sum + e.amount, 0);

  return (
    <div style={{ maxWidth: 480, margin: "2rem auto", fontFamily: "sans-serif" }}>
      <h1>ExpenseWise</h1>
      <input value={title} onChange={(e) => setTitle(e.target.value)} placeholder="Title" />
      <input
        type="number"
        value={amount}
        onChange={(e) => setAmount(e.target.value)}
        placeholder="Amount"
      />
      <select value={category} onChange={(e) => setCategory(e.target.value)}>
        {CATEGORIES.map((c) => <option key={c}>{c}</option>)}
      </select>
      <button onClick={add}>Add</button>
      {error && <p style={{ color: "crimson" }}>{error}</p>}
      <ul>
        {expenses.map((e) => (
          <li key={e._id}>
            {e.title} - Rs. {e.amount} ({e.category}){" "}
            <button onClick={() => remove(e._id)}>x</button>
          </li>
        ))}
      </ul>
      <h3>Total: Rs. {total}</h3>
    </div>
  );
}
```

client/src/main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(<App />);
```

client/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>ExpenseWise</title>
</head>
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

client/package.json

```
{
  "name": "mern-client",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": { "dev": "vite", "build": "vite build" },
  "dependencies": { "react": "^18.3.1", "react-dom": "^18.3.1" },
  "devDependencies": { "@vitejs/plugin-react": "^4.3.0", "vite": "^5.2.0" }
}
```

client/vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({ plugins: [react()] });
```

Output

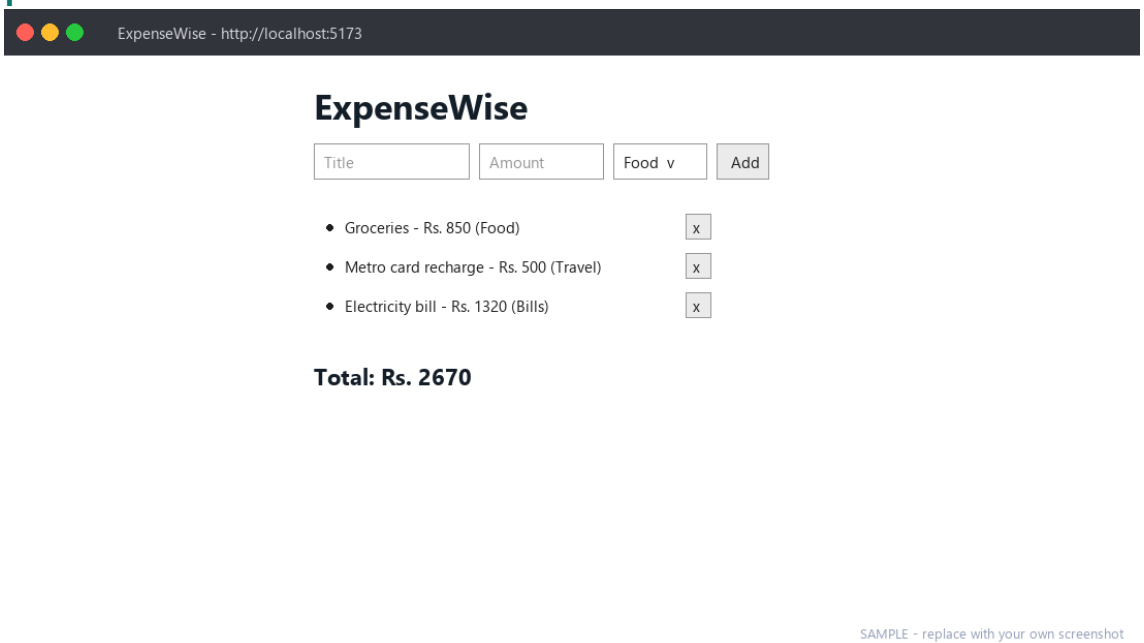


Fig: frontend ui output

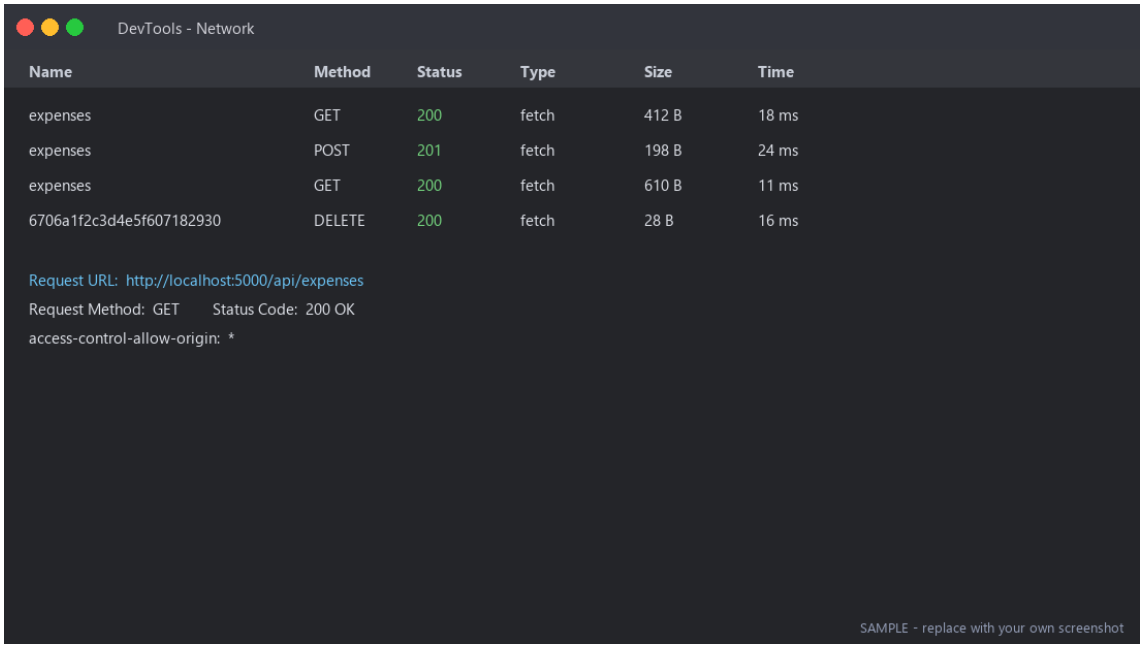


Fig: network tab api call

Result

The program was executed successfully and the expected output was obtained.